

Platform used

I ran the programs on Windows 10 using WSL2. This required use of WSL 2 based engine with Docker Desktop, which is the default on Windows 10.

Versions:

WSL2 (wsl -v):

WSL version: 2.6.1.0

Kernel version: 6.6.87.2-1

WSLg version: 1.0.66

MSRDC version: 1.2.6353

Direct3D version: 1.611.1-81528511

DXCore version: 10.0.26100.1-240331-1435.ge-release

Windows version: 10.0.19045.6332

Ubuntu with WSL2 (lsb_release -a):

Distributor ID: Ubuntu

Description: Ubuntu 22.04.5 LTS

Release: 22.04

Codename: jammy

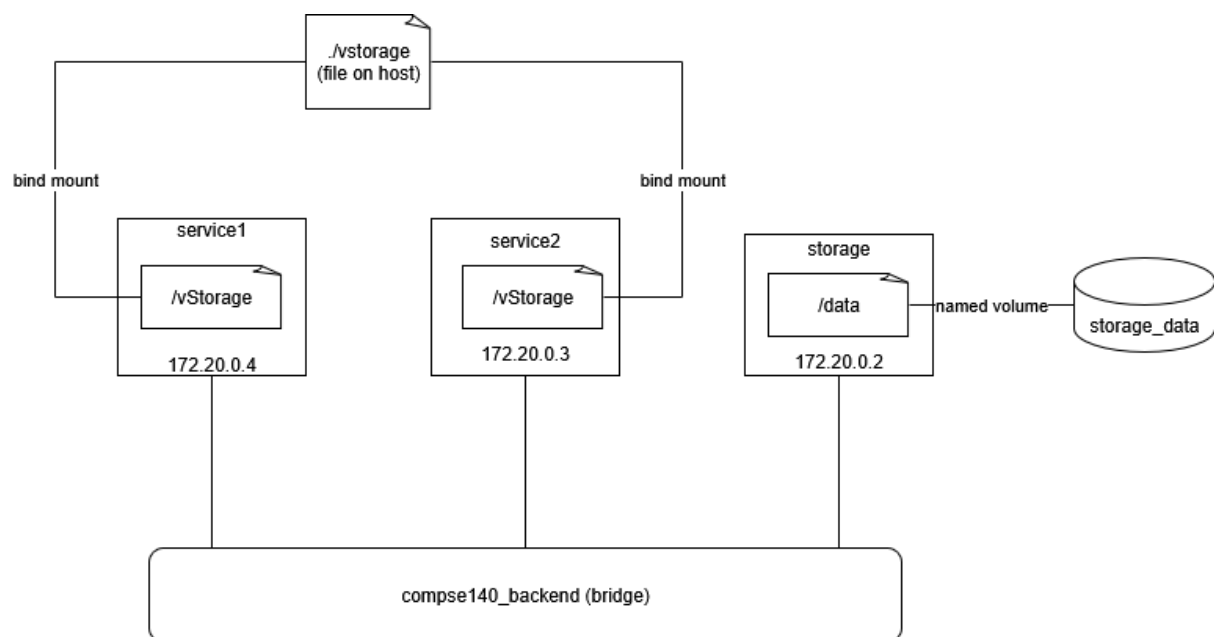
Docker (docker -v):

Docker version 27.3.1, build ce12230

Docker-Compose (docker-compose -v):

Docker Compose version v2.39.2-desktop.1

A diagram showing the services, network and storage:



IP addresses obtained by running command:

```
docker inspect -f
'{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' <container ID>
```

Analysis of the content of the status records:

The changes to disk space caused by text input are so small that I had to make several requests for it to reflect in shown available space. Therefore I cut off the start of the print since it showed an identical free disk amount.

```
topi@DESKTOP-URCV6S7:~/comp.se.140$ cat ./vstorage
...
2025-09-26T15:54:05Z: uptime 0.04489409507555555 hours, free disk in root: 998983.72265625 MBytes
2025-09-26T15:54:05Z: uptime 0.04488269683166664 hours, free disk in root: 998983.72265625 MBytes
2025-09-26T15:54:09Z: uptime 0.04602367042888885 hours, free disk in root: 998983.72265625 MBytes
2025-09-26T15:54:09Z: uptime 0.04601228808416686 hours, free disk in root: 998983.72265625 MBytes
2025-09-26T15:54:10Z: uptime 0.04621745613805556 hours, free disk in root: 998983.71484375 MBytes
2025-09-26T15:54:10Z: uptime 0.0462063598500001 hours, free disk in root: 998983.71484375 MBytes
2025-09-26T15:54:11Z: uptime 0.04632724917027778 hours, free disk in root: 998983.71484375 MBytes
2025-09-26T15:54:11Z: uptime 0.04631586597250008 hours, free disk in root: 998983.71484375 MBytes
topi@DESKTOP-URCV6S7:~/comp.se.140$ curl http://localhost:8199/log
...
2025-09-26T15:54:05Z: uptime 0.04489409507555555 hours, free disk in root: 998983.72265625 MBytes
2025-09-26T15:54:05Z: uptime 0.04488269683166664 hours, free disk in root: 998983.72265625 MBytes
2025-09-26T15:54:09Z: uptime 0.04602367042888885 hours, free disk in root: 998983.72265625 MBytes
2025-09-26T15:54:09Z: uptime 0.04601228808416686 hours, free disk in root: 998983.72265625 MBytes
2025-09-26T15:54:10Z: uptime 0.04621745613805556 hours, free disk in root: 998983.71484375 MBytes
2025-09-26T15:54:10Z: uptime 0.0462063598500001 hours, free disk in root: 998983.71484375 MBytes
2025-09-26T15:54:11Z: uptime 0.04632724917027778 hours, free disk in root: 998983.71484375 MBytes
2025-09-26T15:54:11Z: uptime 0.04631586597250008 hours, free disk in root: 998983.71484375 MBytes
```

The measured uptime is the actual uptime of the applications from when the backends are launched. For this I used two solutions. With service1 (JavaScript) calling `process.uptime()` is sufficient. Service2 (Ruby) stores the timestamp at launch and subtracts it from current time.

The displayed “free disk in root” is the free space reported by the WSL2 Linux distribution I am using, e.g. the theoretical maximum capacity of the virtual hard disk (VHD). This can also be checked from Windows side:

```
PS C:\Users\morso\Code\comp.se.140> wsl df -h /.
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd        1007G  46G  911G   5% /
```

This does not actually reflect the amount of free space on my Windows drive, the one terabyte seems to have been “allocated” to the virtual disk by default. In this context, the measurements are misleading since Linux just reports the amount of space that the virtual disk claims it holds. In a context where the space occupied is substantially increased on each request, some problems would occur once the virtual drive’s contents exceed the actual Windows hard drive space. Some space should be reserved solely for the VM, and the virtual disk space should reflect the reserved space.

Analysis and comparison of the persistent storage solutions:

Using bind mount to tie file `./vstorage` to a docker location:

Accessibility is a plus. You can quickly open the content yourself and see what the data holds. However, this also makes it easier to even accidentally edit the contents, so that the data doesn't reflect the applications history. This solution can be useful during development for quick inspection.

A named volume connected to a location within container:

Docker tooling allows for listing, removing and backing up volumes. A named volume is also safer than a bind mount, editing from the host directly is not possible. However, there is less transparency, and user must open the container in terminal and navigate to the path to browse the contents. For production environments, a named volume is preferred over the two.

Clearing persistent storages:

Remove docker containers and remove the persistent volume that storage-service is using:

```
docker-compose down  
docker volume rm compse140_storage_data
```

Navigate to project root folder. Clear the local vstorage-file:

```
> vstorage
```

Re-run the application with docker-compose up to use fresh data.

What was difficult?

My goal was to make the functionality of both services as identical as possible. This required looking up resources online until I landed on solutions that allowed me to make identical records on both ends that measured the same things. Also, at first it was not clear for me why the measured storage was so large in capacity which required research.

What were the main problems?

At I had some difficulties running the Docker containers with WSL2. I spent some time debugging the issues, until updating Docker Desktop fixed the problems immediately. The actual implementation of Docker containers was quite straightforward since I could mostly just use ready-made Dockerfiles and I have created containers for these environments in the past.